Student Management System using OOP in Python

ARITRA TALUKDAR
CSB24034

April 29, 2026

1 Objective

To design and implement a Student Management System in Python demonstrating key Object-Oriented Programming (OOP) concepts such as:

- Composition
- Encapsulation using `@property`
- Inheritance
- Method Overriding
- Mutable behavior

2 Problem Statement

Design a system with:

- **Address class**: street, city, zipCode
- **Student class**: name, age, address, course list
- Age should be protected and validated using `@property`
- Methods: `add_course()`, `display()`
- Extend Student into **ScholarshipStudent**

3 Class Design

3.1 Address Class

Stores address details and provides a display method.

3.2 Student Class

- Contains name, age, address, and courses
- Uses composition (HAS-A relationship with Address)
- Age is validated using property

3.3 ScholarshipStudent Class

- Inherits from Student
- Adds scholarship amount
- Overrides display method

4 Implementation

4.1 Python Code

```
# Address Class -> Represents a student's address
class Address:
    def __init__(self, street, city, zip_code):
        # Initialize address attributes
        self.street = street
        self.city = city
        self.zip_code = zip_code

    def display(self):
        # Returns formatted address string
        return f"{self.street}, {self.city} - {self.zip_code}"


# Student Class -> Main class
class Student:
    def __init__(self, name, age, address, courses=None):
        self.name = name  # Public attribute

        # Protected attribute (_age) -> should not be accessed directly
        self._age = None

        # Setter is used -> ensures validation happens
        self.age = age

        # Composition -> Student HAS-A Address object
        self.address = address

        # Mutable list handling
        # If no list is passed, create a new list (avoids shared reference issue)
```

```
30          if courses is None:
31              self.courses = []
32          else:
33              self.courses = courses
34
35      # Getter for age
36      @property
37      def age(self):
38          return self._age
39
40      # Setter for age with validation
41      @age.setter
42      def age(self, value):
43          # Validation logic
44          if not isinstance(value, int) or value <= 0 or value >
                120:
45              raise ValueError("Age must be between 1 and 120")
46
47          # Assign valid value
48          self._age = value
49
50      # Method to add course -> modifies same list (mutable
            behavior)
51      def add_course(self, course):
52          self.courses.append(course)
53
54      # Display student details
55      def display(self):
56          print(f"Name: {self.name}")
57          print(f"Age: {self.age}")
58
59          # Calling Address class method -> shows composition usage
60          print(f"Address: {self.address.display()}")
61
62          print("Courses:", ", ".join(self.courses))
63
64
65  # Derived Class -> Inheritance
66  class ScholarshipStudent(Student):
67      def __init__(self, name, age, address, courses,
            scholarship_amount):
68
69          # Call parent constructor -> avoids rewriting code
70          super().__init__(name, age, address, courses)
71
72          # New attribute specific to child class
73          self.scholarship_amount = scholarship_amount
74
75      # Method overriding
76      def display(self):
77          # Call parent display method first
```

```
        super().display()

        # Add extra detail
        print(f"Scholarship Amount: Rs{self.scholarship_amount}")


# --------------------------------
# Testing the system
# --------------------------------

# Create Address object
addr = Address("XYZ ROAD", "TEZPUR", "784001")
addr2 = Address("ABC STREET", "GUWAHATI", "781001")

# Create Student object
student1 = Student("Aritra", 20, addr)

# Add courses -> modifies same list (mutable behavior)
student1.add_course("Data Structures")
student1.add_course("Algorithms")

print("=== Student Details ===")
student1.display()

# Add another course -> persists
student1.add_course("Operating Systems")

print("\nAfter adding another course:")
student1.display()

# Create Scholarship Student
scholar = ScholarshipStudent(
    "Priya", 21, addr2, ["DBMS", "OS"], 50000
)

print("\n=== Scholarship Student Details ===")
scholar.display()
```

5 Concepts Demonstrated

5.1 1. Composition

Student contains an Address object:

`self.address = address`

5.2 2. Encapsulation

Age is protected and accessed via property:

```
@property
def age(self):
    return self._age
```

5.3 3. Data Validation

Ensures valid age:

```
if value <= 0 or value > 120:
    raise ValueError
```

5.4 4. Inheritance

ScholarshipStudent inherits from Student:

```
class ScholarshipStudent(Student):
```

5.5 5. Method Overriding

Display method is extended:

```
super().display()
```

5.6 6. Mutable Behavior

Courses list updates persist:

```
self.courses.append(course)
```

6 Conclusion

The system successfully demonstrates core OOP principles. It models real-world relationships using composition and inheritance, ensures data integrity through encapsulation, and highlights Python's handling of mutable objects.

7 Output

7.1 Program Output

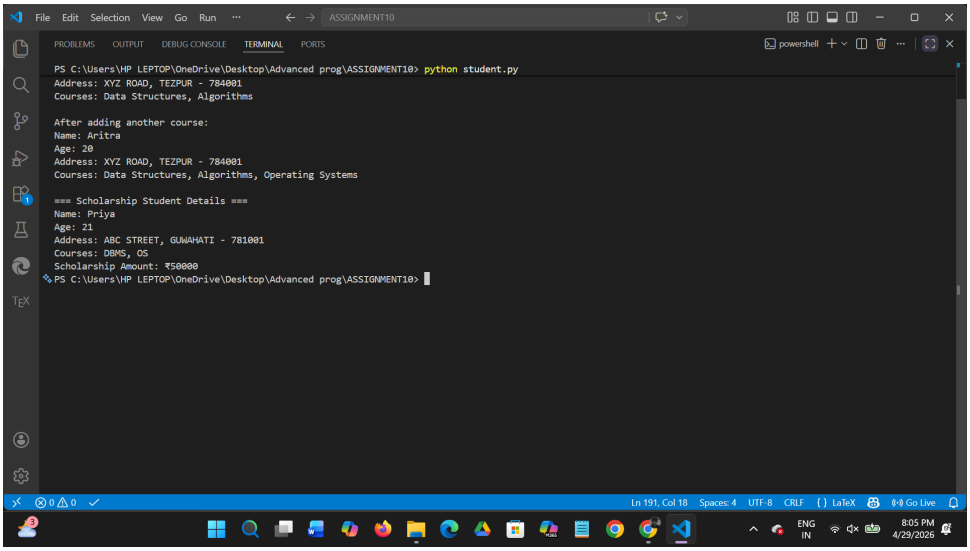

Figure 1: Sample Output